Projeto final de automação IoT

Etapa 1: proposta e planejamento

Engenharia de Software · Internet das Coisas
Católica de Santa Catarina, Centro Universitário
Professor: Msc. Rodrigo Hagstrom

1. Identificação

Nome provisório	Monitor IoT de temperatura para servidor
Disciplina	Internet das Coisas (Engenharia de Software)
Grupo	2
Etapa	1 de 5: proposta e planejamento
Data de entrega	06/11/2026

Integrante
Bruno Ruan dos Santos
Bernardo Wehmuth
João Vítor Dal-Ri
João Vítor Círico
Nicolas Soares Oliveira
Vinícius Bueno de Oliveira

2. Problema e necessidade

Servidores e equipamentos de rede funcionam dentro de uma faixa de temperatura. Acima dela, o equipamento perde desempenho, trava ou desliga. A ASHRAE recomenda manter a entrada de ar de equipamentos de TI entre 18 e 27 °C (diretrizes térmicas do comitê TC 9.9).

O problema identificado pela equipe aparece em salas pequenas, armários de rede e laboratórios. Nesses lugares, a temperatura costuma ser conferida só quando alguém percebe uma falha. A ventoinha, quando existe, fica ligada o tempo todo ou depende de alguém acioná-la, e não há registro que mostre quando e por que a temperatura subiu.

A necessidade é medir a temperatura de forma contínua, acionar a ventilação sem intervenção humana, avisar quando a situação piora e guardar o histórico para análise.

3. Público, usuário e cenário de aplicação

O usuário é a pessoa responsável pela infraestrutura de TI de um laboratório, de uma escola ou de uma pequena empresa, que mantém poucos servidores ou equipamentos de rede em uma sala ou em um rack.

No protótipo, o rack é representado por um gabinete pequeno, como uma caixa, com o sensor e a ventoinha dentro. O operador acompanha o painel no computador e pode alterar os limites e o modo de operação à distância.

4. Objetivos

Objetivo geral. Construir um protótipo IoT que monitora a temperatura de um servidor ou rack, aciona a ventilação automaticamente, avisa o operador e disponibiliza os dados em um painel remoto, com análises simples do histórico.

Objetivos específicos:

1. Medir a temperatura do gabinete com um sensor DS18B20 e, como reforço, a umidade com o DHT11 do kit.
2. Classificar a situação em três estados (normal, atenção e crítico), com histerese, e acionar ventoinha, LED RGB e buzzer conforme o estado.
3. Enviar a telemetria por Wi-Fi, via MQTT, em intervalos regulares.
4. Exibir os dados em um painel web e guardar o histórico em um banco SQLite.
5. Receber comandos remotos para alterar os limites e alternar entre modo automático e manual.
6. Calcular, sobre o histórico, a taxa de variação da temperatura e a previsão do tempo até o limite crítico (camada de reforço).

5. Funcionamento esperado da automação

O ESP32 lê os sensores a cada ciclo e compara a temperatura com dois limites, atenção e crítico. O estado define a ação local, que não depende da rede: cor do LED RGB, ventoinha e buzzer.

Estado	Entra quando	Sai quando	LED RGB	Ventoinha	Buzzer
Normal	Abaixo de 27 °C na partida, ou 25 °C ou menos ao voltar de atenção	Sobe a 27 °C ou mais	Verde	Desligada	Desligado
Atenção	27 °C ou mais	Cai a 25 °C ou menos (volta a normal) ou sobe a 32 °C ou mais (crítico)	Amarelo	Ligada	Desligado
Crítico	32 °C ou mais	Cai a 30 °C ou menos (volta a atenção)	Vermelho	Ligada	Ligado

A histerese evita que a ventoinha ligue e desligue repetidamente quando a temperatura oscila perto de um limite: o sistema só volta ao estado anterior depois que a temperatura cai 2 °C abaixo do limite que o ativou. Os valores de 27, 32, 25 e 30 °C são iniciais. Serão calibrados na bancada e poderão ser alterados pelo painel.

A cada 5 segundos, o ESP32 publica uma mensagem JSON com temperatura, umidade, estado da tampa do gabinete (lido pelo LDR), estado da ventoinha e modo de operação. O servidor, escrito em Python, recebe a mensagem, registra data e hora, grava no banco SQLite e atualiza o painel web. Os comandos feitos no painel voltam ao ESP32 pelo mesmo broker. Se o Wi-Fi cair, o ESP32 mantém o controle local e tenta reconectar sozinho. Um botão silencia o buzzer.

6. Componentes

A lista é preliminar. O ESP32 e o Kit Iniciante RoboCore já estão disponíveis. Os demais itens serão providenciados.

Componente	Função	Situação
ESP32 (placa de desenvolvimento)	Microcontrolador com Wi-Fi	Já temos
DHT11 e resistor de 10 kΩ	Umidade e temperatura de apoio	Já temos (kit)
LED RGB e 3 resistores de 300 Ω	Indica o estado do sistema	Já temos (kit)
Buzzer	Alarme sonoro	Já temos (kit)
LDR e resistor de 10 kΩ	Detecta a abertura da tampa pela entrada de luz	Já temos (kit)
Botão	Silencia o alarme	Já temos (kit)
Protoboard, jumpers e resistores	Montagem do circuito	Já temos (kit e laboratório)
DS18B20 e resistor de 4,7 kΩ	Sensor de temperatura principal	Providenciar
Ventoinha de 5 V	Resfriamento do gabinete	Providenciar
Transistor NPN (BC337 ou 2N2222) e diodo 1N4007	Acionar a ventoinha sem sobrecarregar o GPIO	Providenciar
Capacitor eletrolítico de 100 µF	Estabilizar a alimentação da ventoinha, se o ESP32 reiniciar	Providenciar, se necessário

O DHT11 tem erro de ±2 °C, segundo o material do kit, o que é alto para detectar variações pequenas. Por isso o DS18B20, com erro de ±0,5 °C no datasheet, será o sensor principal, e o DHT11 fica como apoio para a umidade. Se o resistor de 4,7 kΩ faltar, dois resistores de 10 kΩ em paralelo (cerca de 5 kΩ) servem.

Software e serviços: firmware em C++ no Arduino IDE, com as bibliotecas OneWire, DallasTemperature, DHT, PubSubClient e ArduinoJson; simulador Wokwi; servidor em Python (FastAPI e paho-mqtt, em ambiente virtual) com banco SQLite e painel web em HTML e JavaScript; broker MQTT na nuvem; repositório no GitHub.

7. Arquitetura conceitual inicial

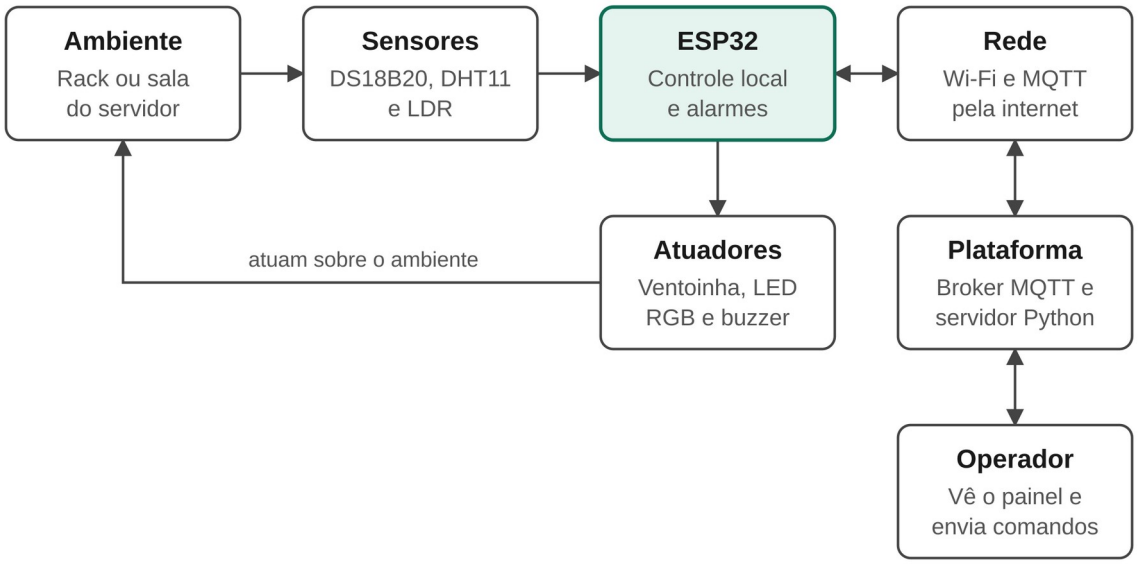


Figura 1. Arquitetura conceitual: ambiente, sensores, ESP32, rede, plataforma e atuadores.

O dado percorre o caminho ambiente, sensor, microcontrolador, rede, plataforma e operador. A ação sobre o ambiente tem dois caminhos: o ESP32 aciona os atuadores diretamente, como controle local, e o operador pode enviar comandos pela plataforma.

8. Viabilidade e requisitos mínimos

Requisito mínimo do projeto IoT	Como o projeto atende
Microcontrolador compatível	ESP32 com Wi-Fi integrado.
Ao menos um sensor relacionado ao problema	DS18B20 para temperatura, com DHT11 e LDR de apoio.
Ao menos uma saída física	Ventoinha, LED RGB e buzzer.
Lógica de automação baseada nos dados	Estados normal, atenção e crítico com histerese.
Comunicação em rede com plataforma ou serviço	Wi-Fi, MQTT e servidor em Python.
Telemetria, eventos ou registros	Mensagens JSON periódicas, painel web e histórico em banco SQLite.
Evidências verificáveis	Tabela de evidências mantida desde esta etapa (seção 11).

O escopo está dividido em três camadas para caber no prazo.

Camada	Itens	Quando entra
Núcleo	DS18B20, ventoinha com transistor, LED RGB, buzzer, histerese, MQTT, reconexão, painel web, banco SQLite e comandos de modo e limite	Obrigatório
Reforço	DHT11 (umidade), LDR (tampa aberta), botão para silenciar, status online e offline, taxa de variação e previsão do tempo até o limite	Se as Etapas 3 e 4 fecharem no prazo
Bônus	Alertas no Telegram, outros indicadores sobre o histórico e um segundo DS18B20 para medir a diferença entre entrada e saída de ar	Somente se sobrar tempo

A viabilidade depende do tempo de bancada, porque as etapas são feitas em aula. Para reduzir esse risco, o circuito e o código serão testados antes no simulador Wokwi, e a aula fica para montar e validar o protótipo físico.

9. Divisão de responsabilidades

Frente	Atividades	Responsável	Apoio
Hardware e gabinete	Circuito, protoboard, gabinete (possivelmente impresso em 3D) e testes elétricos	Bruno	Vinícius
Firmware	Leitura dos sensores, estados, MQTT e reconexão	Bernardo	João Vítor Dal-Ri
Servidor, banco e painel	Servidor em Python, banco SQLite, painel web e indicadores	João Vítor Círico	Nicolas
Documentação e testes	Documentos, tabela de evidências, repositório e plano de testes	Vinícius	Todos

Os papéis acima são provisórios e podem mudar a cada entrega. Na prática, todos os integrantes ajudam em todas as frentes, e qualquer um pode ser questionado sobre qualquer parte do projeto na defesa.

10. Cronograma e riscos

Etapa	Entrega	No simulador, antes da aula	Na bancada, durante a aula
1	Proposta e planejamento (06/11)	Projeto base no Wokwi com ESP32 e leitura de um sensor	Inventário do kit, teste de Wi-Fi no laboratório e pedido das peças
2	Projeto técnico e arquitetura (06/11)	Circuito completo e tabela de pinos	Foto da placa, para mapear os pinos na montagem
3	Hardware e firmware (13/11)	Firmware com estados e histerese, sem rede	Montagem na protoboard, componente por componente, e teste do transistor com a ventoinha isolado
4	Integração IoT e controle remoto (20/11)	MQTT e servidor Python recebendo os dados	Wi-Fi do laboratório, comandos remotos e teste de reconexão
5	Validação e documentação (27/11)	Cálculo dos indicadores com dados simulados	Testes com aquecimento do sensor e abertura da tampa, e plano de testes

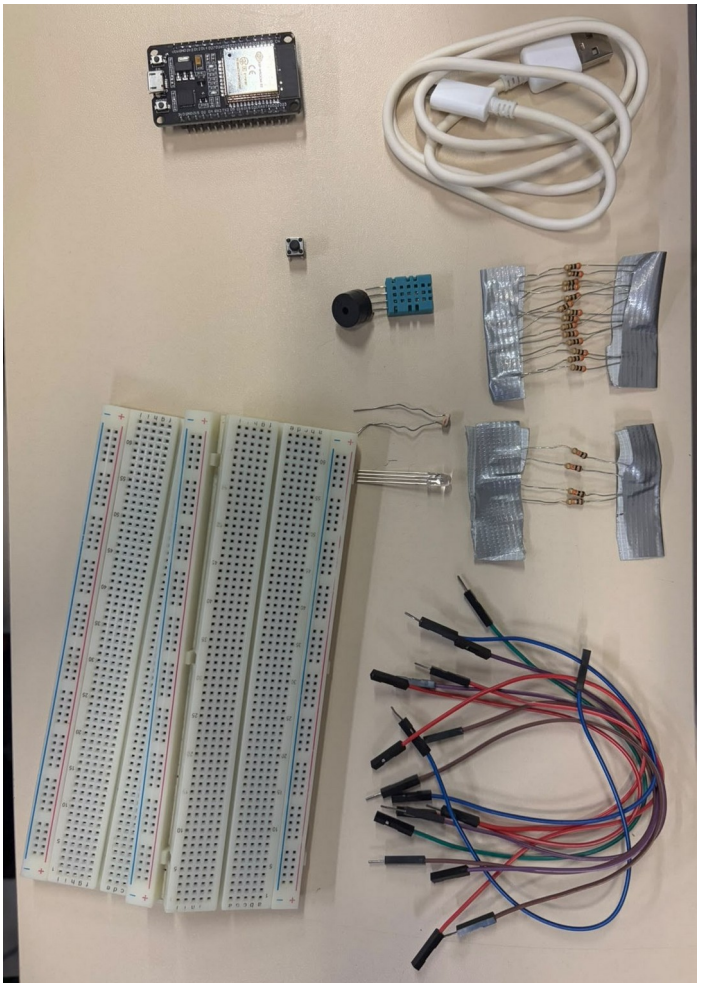
Riscos identificados:

Risco	Efeito	Resposta
As peças a providenciar atrasam	A Etapa 3 fica sem o DS18B20 ou a ventoinha	Fazer o pedido já na Etapa 1. Usar o DHT11 e um LED no lugar da ventoinha como solução provisória.
O Wi-Fi do laboratório bloqueia a saída para o broker	A Etapa 4 fica sem comunicação	Testar a conexão na Etapa 1 ou 2. Usar o hotspot do celular como alternativa.
O LED RGB fica fraco com 3,3 V	O estado fica pouco visível	Testar cedo. Reduzir os resistores ou trocar por LEDs comuns.
O ESP32 reinicia quando a ventoinha liga	Perda de dados e funcionamento instável	Usar capacitor de 100 µF ou alimentar a ventoinha por uma fonte separada, com GND comum.
O servidor em Python exige mais código do que uma ferramenta pronta	A Etapa 4 atrasa	Começar pelo essencial (receber, gravar e mostrar a temperatura) e deixar os indicadores para depois. Manter o código simples e dividido em poucos arquivos.
Pouco tempo de bancada	Uma parcial não entregue custa 1 ponto	Validar circuito e código no Wokwi antes da aula. Implementar o reforço e o bônus só depois que o núcleo funcionar.

11. Registro de evidências e anexos

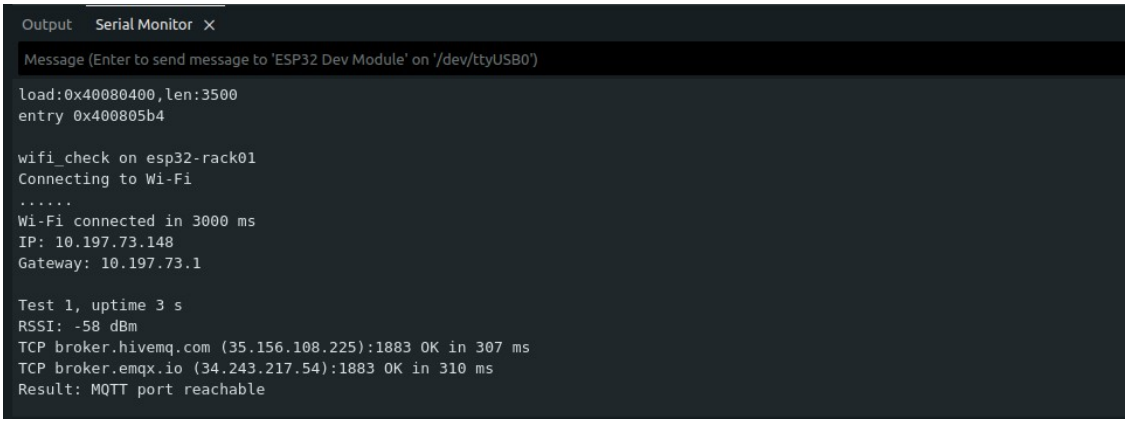
Requisito ou funcionalidade	Tipo de evidência	Arquivo ou link	Como a evidência comprova
Problema, objetivos e cronograma	Documento	Este arquivo, seções 2, 4 e 10	Descreve o problema, os objetivos e o plano das cinco etapas.
Arquitetura conceitual	Diagrama	Figura 1	Mostra o caminho do dado entre ambiente, ESP32, plataforma e atuadores.
Lista de componentes	Tabela	Seção 6	Separa o que já existe do que será providenciado.
Material disponível	Foto	Anexo A e link no OneDrive	Confirma os componentes do kit, o ESP32 e as protoboards do laboratório.
Conexão Wi-Fi no laboratório	Captura do monitor serial	Anexo B e link no OneDrive	Confirma que o ESP32 acessa a rede do laboratório e alcança os brokers MQTT na porta 1883.

Anexo A. Material disponível.



Material disponível na aula de 06/11/2026: ESP32 DevKit V1 com o cabo USB, duas protoboards de 830 pontos do laboratório, botão, buzzer, DHT11, LDR, LED RGB, resistores de 10 kΩ e de 300 Ω e jumpers. Foto no OneDrive: https://catolicasc-my.sharepoint.com/:i:/g/personal/vinicius06_oliveira_catolicasc_edu_br/IQD52qGG6Z5wQo8l5tFILaguAcQWLbNotDC3mmELsvolHtE?e=sX8HvR.

Anexo B. Conexão Wi-Fi no laboratório.



Recorte do monitor serial durante o teste de Wi-Fi, com o sketch gravado no ESP32 na aula de 06/11/2026. A placa conectou à rede da sala em 3 segundos, com sinal de -58 dBm, e os dois brokers MQTT, HiveMQ e EMQX, responderam na porta 1883. Numa execução anterior, de cerca de 4 minutos, o sinal chegou a -92 dBm e uma conexão ao HiveMQ falhou, mas o EMQX respondeu no mesmo teste. A captura completa está no OneDrive: https://catolicasc-my.sharepoint.com/:i:/g/personal/vinicius06_oliveira_catolicasc_edu_br/IQANuDwvGrAAR7qTAUxoq4zSAdt3VoF2efG5S9E0a9mr57c?e=dYv7cy.